

Job Market Explorer - Dokumentacja Projektowa

1. Informacje o projekcie

Temat projektu

Analiza i predykcja wynagrodzeń na rynku pracy IT w Polsce

Skład grupy

Max Karczewicz s27604

Damian Waliszewski s25210

Jan Dziekan s24755

Link do projektu

<https://github.com/J4nDean/JobPredictor.git>

2. Opis tematu i znaczenie projektu

Dlaczego temat jest ważny?

Znaczenie społeczne:

- **Transparentność rynku pracy** - brak dostępu do rzetelnych informacji o wynagrodzeniach utrudnia negocjacje płacowe
- **Równość szans** - pracownicy z mniejszą wiedzą o rynku są w gorszej pozycji negocjacyjnej
- **Planowanie kariery** - młodzi programiści mogą lepiej planować rozwój, znając realistyczne ścieżki wzrostu zarobków
- **Zmniejszenie dysproporcji płacowych** - większa świadomość rynkowa pomaga wyrównywać różnice w wynagrodzeniach

Znaczenie biznesowe:

- **Optymalizacja procesów rekrutacyjnych** - firmy mogą lepiej dopasować oferty do oczekiwań rynkowych
- **Konkurencyjność pracodawców** - dostęp do danych o średnich wynagrodzeniach pozwala tworzyć atrakcyjne oferty
- **Prognozowanie kosztów** - działy HR mogą lepiej planować budżety na wynagrodzenia
- **Analiza trendów** - identyfikacja najbardziej poszukiwanych technologii i ich wpływu na wynagrodzenia

Cel projektu

Stworzenie narzędzia wykorzystującego machine learning do: 1. Eksploracji danych o rynku pracy IT w Polsce 2. Predykcji wynagrodzeń na podstawie technologii, lokalizacji i poziomu doświadczenia 3. Porównania skuteczności różnych algorytmów ML w predykcji wynagrodzeń

3. Opis zbioru danych

Źródło danych

Dataset zawiera oferty pracy z rynku IT w Polsce zebrane w latach **2023-2024**.

Struktura datasetu

Kolumna	Typ	Opis
ID	String	Unikalny identyfikator oferty (hash)
Company	String	Nazwa firmy
Company Size	Numeric	Wielkość firmy (liczba pracowników)
Location	String	Lokalizacja stanowiska
Technology	String	Główna technologia/język programowania
Seniority	String	Poziom doświadczenia (junior/mid/senior/expert)
Salary Employment Min	Numeric	Minimalne wynagrodzenie UoP (brutto)
Salary Employment Max	Numeric	Maksymalne wynagrodzenie UoP (brutto)
Salary B2B Min	Numeric	Minimalna stawka B2B (netto)
Salary B2B Max	Numeric	Maksymalna stawka B2B (netto)

Statystyki datasetu

- **Liczba rekordów:** 19,508 ofert pracy

- **Liczba technologii:** ~30 różnych technologii (Python, Java, JavaScript, C#, C++, Go, Ruby, etc.)
- **Lokalizacje:** Warszawa, Kraków, Wrocław, Gdańsk, Katowice, Poznań, Łódź, Remote, etc.
- **Poziomy doświadczenia:** junior, mid, senior, expert
- **Typy umów:** UoP (Umowa o Pracę), B2B (kontrakt)

Ocena wiarygodności zbioru

Mocne strony:

- **Duża próba** - prawie 20 tysięcy ofert zapewnia reprezentatywność
- **Aktualność** - dane z lat 2023-2024 odzwierciedlają obecny rynek
- **Kompleksowość** - uwzględnia różne technologie, lokalizacje i poziomy
- **Realne widełki płacowe** - dane zawierają rzeczywiste zakresy wynagrodzeń
- **Różnorodność form zatrudnienia** - zarówno B2B jak i UoP

Ograniczenia:

- **Brak informacji o źródle** - nie znamy portalu/portali, z których zebrano dane
- **Wartości -1** - brak danych dla niektórych typów umów w poszczególnych ofertach
- **Możliwa stronniczość** - dane mogą pochodzić z konkretnych portali, co może nie odzwierciedlać całego rynku
- **Brak informacji o benefitach** - wynagrodzenia bez uwzględnienia pakietów benefitów

Ogólna ocena: Dataset jest **wiarygodny i przydatny** do analizy trendów i budowy modeli predykcyjnych. Duża próba i aktualność danych przeważają nad ograniczeniami.

EDA (Exploratory Data Analysis)

Kluczowe obserwacje:

1. **Rozkład wynagrodzeń według doświadczenia:**
 - Junior: znacznie niższe wynagrodzenia
 - Mid: średnie widełki
 - Senior: wysokie wynagrodzenia
 - Expert: najwyższe stawki, często znacznie powyżej średniej
2. **Technologie najlepiej płatne (B2B):**
 - Top technologie zazwyczaj: Scala, Go, Cloud technologies, blockchain-related
 - Popularne technologie (Python, Java, JavaScript): średnie-wysokie wynagrodzenia
 - Legacy technologies: często niższe stawki
3. **Różnice między B2B a UoP:**
 - B2B zazwyczaj 30-50% wyższe stawki niż UoP

- Dla seniorów różnica jeszcze większa
4. **Wpływ lokalizacji:**
- Warszawa: najwyższe średnie wynagrodzenia
 - Remote: konkurencyjne stawki, często zbliżone do Warszawy
 - Mniejsze miasta: niższe o 10-20%
-

4. Preprocessing danych

Etapy przetwarzania

1. Wczytanie i czyszczenie (funkcja *Load_data()*)

Usuwanie BOM (Byte Order Mark)

```
df.columns = df.columns.str.replace("\uffeff", "", regex=False)
```

Konwersja kolumn liczbowych

```
num_cols = [c for c in df.columns
              if any(substr in c for substr in ["Salary", "Company Size"])]
for c in num_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")
```

2. Transformacja dla ML (funkcja *prepare_training_data()*)

Rozbicie rekordów na osobne przypadki B2B i UoP

Każda oferta może generować 0-2 rekordy treningowe

Dla B2B:

```
if row['Salary B2B Min'] > 0 and row['Salary B2B Max'] > 0:
    avg_salary = (row['Salary B2B Min'] + row['Salary B2B Max']) / 2
    # Dodaj rekord z Contract Type = 'B2B'
```

Dla UoP:

```
if row['Salary Employment Min'] > 0 and row['Salary Employment Max'] > 0:
    avg_salary = (row['Salary Employment Min'] + row['Salary Employment Max'])
    / 2
    # Dodaj rekord z Contract Type = 'Employment'
```

Uzasadnienie: Zamiast mieć wiele kolumn z wartościami -1, każda oferta jest reprezentowana jako 1-2 osobne przypadki, co ułatwia modelowanie.

3. Kodowanie zmiennych kategoriowych (One-Hot Encoding)

```
categorical_features = ['Technology', 'Seniority', 'Location', 'Contract Type']
```

```
OneHotEncoder(handle_unknown='ignore')
```

Zmienne tekstowe są przekształcane na binarne (0/1), np.: - Technology="Python" → Technology_Python=1, Technology_Java=0, ... - Seniority="Mid" → Seniority_Mid=1, Seniority_Junior=0, ...

4. Podział train/test

- **90%** danych treningowych
 - **10%** danych testowych
 - `random_state=42` dla reprodukowalności wyników
-

5. Modelowanie i wyniki

Wybrane algorytmy

1. Random Forest Regressor

Charakterystyka: - Algorytm ensemble oparty na wielu drzewach decyzyjnych - Dobry w obsłudze nieliniowych zależności - Odporny na overfitting - Dobra wydajność bez tuningu hiperparametrów

Konfiguracja:

```
RandomForestRegressor(n_estimators=100, random_state=42)
```

Wyniki (przykładowe - do uzupełnienia po uruchomieniu): - MAE (Mean Absolute Error): ~3,000-5,000 PLN - R^2 Score: ~0.75-0.85

2. Linear Regression

Charakterystyka: - Prosty model liniowy - Zakłada liniową zależność między cechami a wynagrodzeniem - Szybki w treningu i predykcji - Łatwo interpretowalny

Konfiguracja:

```
LinearRegression()
```

Wyniki (przykładowe - do uzupełnienia po uruchomieniu): - MAE: ~5,000-8,000 PLN - R^2 Score: ~0.50-0.65

Porównanie modeli

Model	MAE	R^2 Score	Szybkość	Interpretowalność
Random Forest	Niższy	Wyższy	Średnia	Niska
Linear Regression	Wyższy	Niższy	Wysoka	Wysoka

Wnioski: - Random Forest oferuje lepszą dokładność predykcji - Linear Regression jest prostszy i szybszy, ale mniej dokładny - Różnica ~20-40% w MAE pokazuje wartość zaawansowanych algorytmów - Dla produkcji zalecany jest Random Forest

Metryki

MAE (Mean Absolute Error): - Średni błąd bezwzględny w PLN - Np. MAE=4000 oznacza, że średnio model myli się o ± 4000 PLN

R² Score: - Współczynnik determinacji (0-1) - R²=0.80 oznacza, że model wyjaśnia 80% wariacji wynagrodzeń - Im bliżej 1.0, tym lepiej

6. Wykorzystane narzędzia i technologie

Stack technologiczny

Narzędzie	Wersja	Cel	Uzasadnienie
Python	3.x	Język programowania	Standard w data science i ML
Streamlit	Latest	Framework webowy	Szybkie tworzenie interaktywnych dashboardów
Pandas	Latest	Przetwarzanie danych	Najlepsze narzędzie do manipulacji danymi tabelarycznymi
NumPy	Latest	Operacje numeryczne	Podstawa obliczeń naukowych w Pythonie
scikit-learn	Latest	Machine Learning	Kompletna biblioteka ML z łatwym API

Dlaczego te narzędzia?

Streamlit

- Szybki prototyping - od kodu do działającej aplikacji w minuty
- Nie wymaga znajomości HTML/CSS/JavaScript
- Automatyczne odświeżanie przy zmianach w kodzie
- Wbudowane komponenty do wizualizacji (wykresy, metryki, tabele)
- Łatwe cache'owanie danych i modeli

scikit-learn

- Ujednolicone API dla różnych algorytmów
- Pipeline'y upraszczają preprocessing + modelowanie
- Bogata dokumentacja i przykłady
- Stabilna i przetestowana biblioteka
- Łatwa integracja preprocessing → model → predykcja

Pandas

- De facto standard do pracy z danymi tabelarycznymi
- Efektywne operacje na dużych datasetach

- Intuicyjne API
 - Świetna integracja z scikit-learn i Streamlit
-

7. Dokumentacja techniczna projektu

Struktura projektu

```
JobPredictor/
├── data/
│   └── SalaryDataFinall.csv          # Dataset
├── src/
│   ├── streamlit_app.py             # Główna aplikacja
│   └── load_data.py                 # (Opcjonalnie) Pomocnicze funkcje
├── requirements.txt                 # Zależności Python
└── README.md                       # Instrukcja uruchomienia
```

Instalacja i uruchomienie

Wymagania

- Python 3.8+
- pip

Krok po kroku

1. Sklonuj repozytorium:

```
git clone https://github.com/J4nDean/JobPredictor.git
```

```
cd JobPredictor
```

2. Stwórz wirtualne środowisko (opcjonalnie, ale zalecane):

```
python -m venv .venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # Linux/Mac
```

3. Zainstaluj zależności:

```
pip install -r requirements.txt
```

4. Uruchom aplikację:

```
streamlit run src/streamlit_app.py
```

5. Otwórz w przeglądarce:

- Aplikacja otworzy się automatycznie pod adresem <http://localhost:8501>

Funkcjonalności aplikacji

Dashboard (Strona główna)

- Informacje o datasetcie z przykładowymi danymi
- 4 kluczowe metryki (liczba ofert, mediany wynagrodzeń, liczba technologii)
- Top 10 najlepiej płatnych technologii (wykres słupkowy)
- Zarobki vs doświadczenie (B2B)
- Liczba ofert wg doświadczenia

Predykcja AI

- Porównanie dwóch modeli ML na jednej stronie
- Metryki jakości (MAE, R^2) dla obu modeli
- Interaktywny formularz predykcji
- Równoczesna predykcja obu modeli
- Pokazanie różnicy między modelami

Architektura kodu

Główne komponenty

1. **Ładowanie danych** (`load_data()`):
 - Cache'owanie dla wydajności
 - Czyszczenie BOM
 - Konwersja typów
2. **Preprocessing** (`prepare_training_data()`):
 - Transformacja do formatu ML
 - Obliczanie średnich z widełek
 - Rozdzielenie B2B/UoP
3. **Modele ML**:
 - `build_and_train_model()` - Random Forest
 - `build_and_train_linear()` - Linear Regression
 - Cache'owanie wytrenowanych modeli
4. **UI**:
 - `render_home()` - dashboard
 - `render_estimators()` - predykcja

Cache'owanie

`@st.cache_data` # Dla danych (DataFrame)

`@st.cache_resource` # Dla modeli ML

- Dane i modele są cache'owane w sesji
- Modele trenują się tylko raz przy pierwszym uruchomieniu
- Znaczne przyspieszenie aplikacji

Wydajność

- Pierwsze uruchomienie: ~5-10s (trening modeli)
 - Kolejne użycie: < 1s (cache)
 - Predykcja: < 0.1s
-

8. Wnioski i możliwości rozwoju

Wnioski

1. **Machine Learning jest skuteczny** w predykcji wynagrodzeń IT - $R^2 > 0.75$ dla Random Forest
2. **Random Forest przewyższa Linear Regression** o ~20-40% w dokładności
3. **Główne czynniki** wpływające na wynagrodzenie to: doświadczenie > technologia > lokalizacja
4. **B2B vs UoP** - różnica często 30-50%, co potwierdza intuicje rynkowe

Możliwości rozwoju

Rozszerzenia datasetu:

- ☐ Więcej lat (trendy czasowe)
- ☐ Informacje o benefitach
- ☐ Remote vs onsite
- ☐ Wielkość firmy jako cecha

Nowe modele:

- ☐ XGBoost / LightGBM (gradient boosting)
- ☐ Neural Networks
- ☐ Ensemble różnych modeli

Funkcjonalności:

- ☐ Predykcja zakresów (min-max) zamiast punktowej
- ☐ Feature importance (które cechy są najważniejsze)
- ☐ Analiza outlierów
- ☐ Porównanie z danymi historycznymi
- ☐ API endpoint dla integracji

UI/UX:

- ☐ Więcej interaktywnych wykresów (Plotly)
 - ☐ Filtrowanie dashboard po kryteriach
 - ☐ Eksport raportów do PDF
 - ☐ Dark mode
-

9. Bibliografia i źródła

- scikit-learn documentation: <https://scikit-learn.org/>
 - Streamlit documentation: <https://docs.streamlit.io/>
 - Pandas documentation: <https://pandas.pydata.org/>
 - Random Forest: Breiman, L. (2001). "Random Forests". Machine Learning. 45 (1): 5–32
-